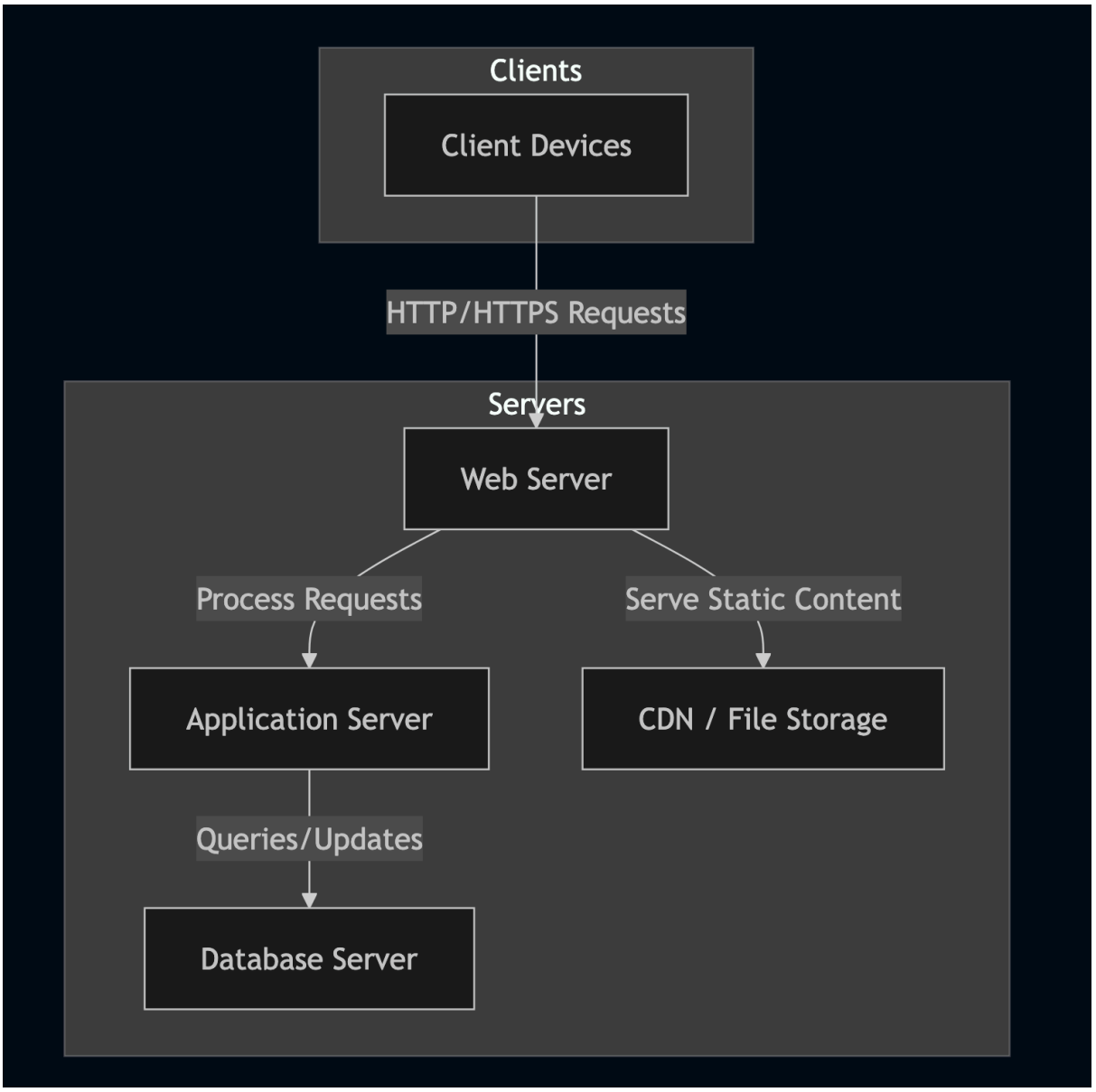
Client/ Server Architecture

When browsing the web, sending an email, streaming Netflix or playing a web-based game, you're interacting with a network-based computing system called Client/ Server architecture which is a method of organising and distributing resources between clients and servers.

The architecture is a model that allows communication and data exchange between applications over a single or multiple server points.

The model is divided into two parts:



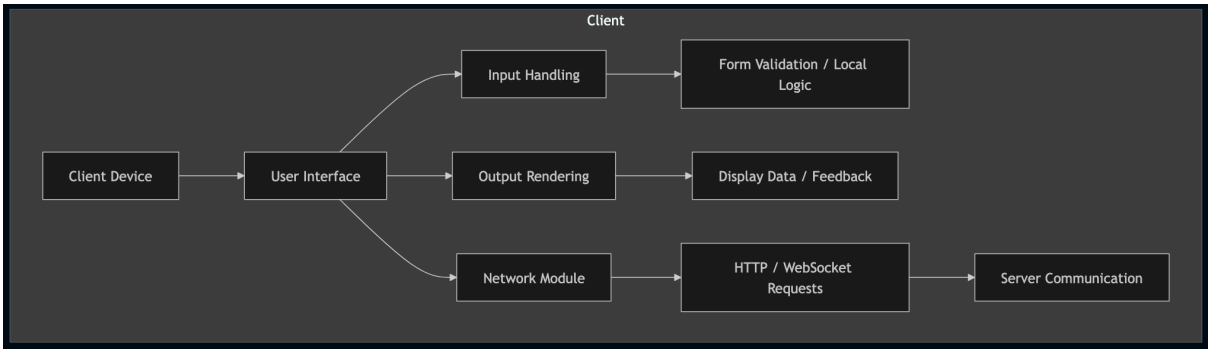
Components and Characteristics of Client/Server Architecture

Client

The client is an application that requests data and/ or services from the server such as storage, calculations and other functions.

A client device is consumer-facing and generally includes web browsers, desktop apps or mobile apps that users interact with. They communicate with the server to delegate tasks, make transactions or retrieve information.

Client and server ends can be different in requirements with hardware and software. They may also come from different vendors. Though, both interact directly with a transport layer protocol to establish communication for sending and receiving data.



Server

The server is an application that processes said client requests and sends responses or performs actions based on those requests. The server and client can both live on the same computer or different devices on the network.

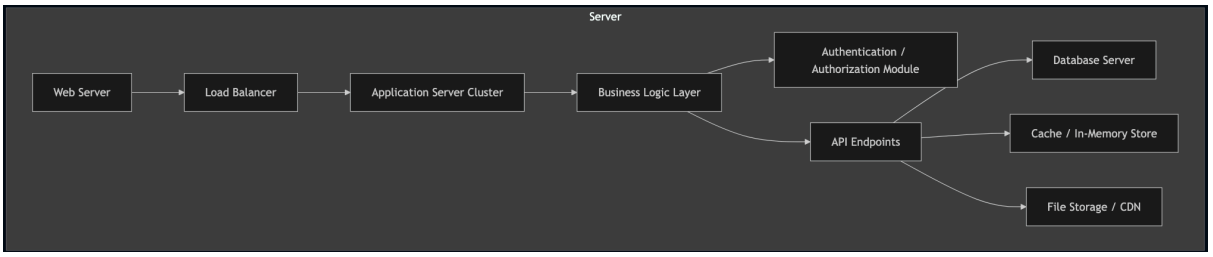
A server offers services or solutions to clients over the network. They can handle processing client requests, which can include tasks such as file storage, database access and application hosting, as well as back end activities such as computations, business logic and data management which reduces what clients need to handle.

A single server can respond with multiple services simultaneously but each service must run it’s own server program.

Some examples of servers are:
File Servers: Upon saving documents on services like Microsoft Office, you are interacting with file servers. This type of server stores data centrally and allow multiple clients to use it.

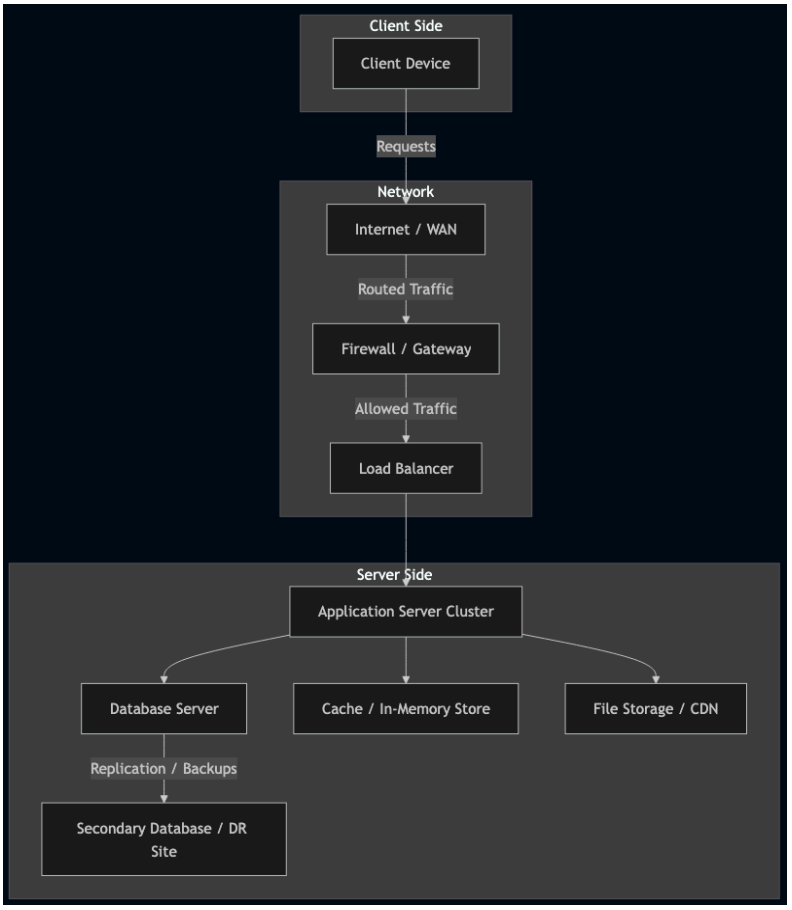
Email Servers: Emails are the primary communication method of businesses due to convenience and speed. Multiple server components work in tandem to deliver emails between users across different mail servers.

Web Servers: Very performant and powerful servers that host websites. Web clients can access these servers through DNS or IP addresses.



Network

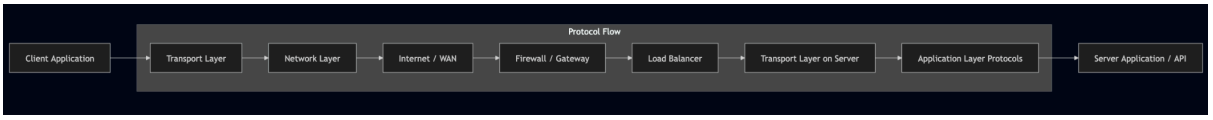
This serves as the channel through which clients and servers are connected for data transfer between them. These can range from local area networks within a single building, to wide area networks and the internet. It acts as the intermediary, facilitating the interchange of requests and responses between the clients and servers, which influences the speed and reliability of these interactions.



Protocol

These are rules that define how data is exchanged between the clients and servers. They ensure the communication is secure, ordered and understandable. Common protocols are HTTP or HTTPS for web services, FTP for file transfers and SMTP for email. They assist in bridging communication between different systems, independent of their technology stack.

Both client and server computers require a full stack of protocols where the transport protocol uses lower layer protocols to send and receive individual messages.



Middleware

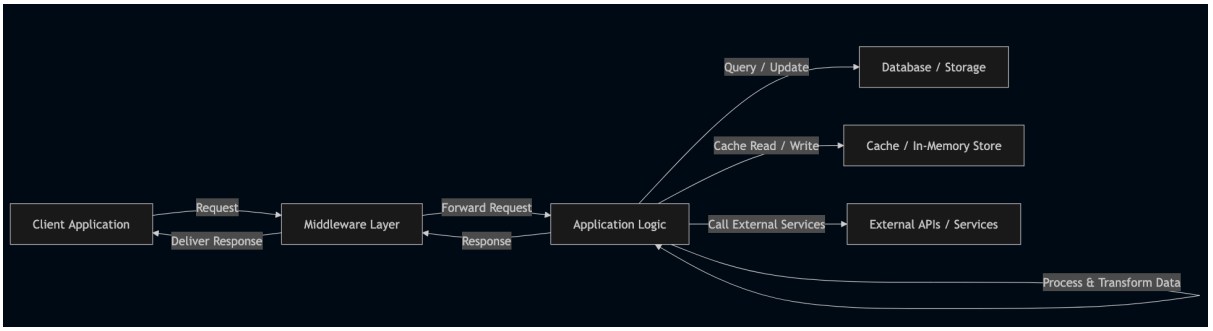
Middleware is a software layer within a server application that intercepts and processes requests and responses during communication between the client and the server. It allows developers to insert reusable logic into the request-response pipeline, enabling tasks such as authentication, logging, data transformation, and error handling before the request reaches the core application logic.

Additionally, middleware can enforce authorization by ensuring that only clients with appropriate permissions can access certain resources or perform specific operations. It can also validate incoming requests, confirming that data conforms to expected formats, types, and business rules before it is processed. This helps reduce errors, improve security, and maintain the integrity of the application.

Application Logic

This is the processes and code that determine how the server responds to requests involving big data processing, business rules and workflows on the server side. It makes sure the server correctly interprets client requests, performs data manipulations or necessary calculations and delivers appropriate responses.

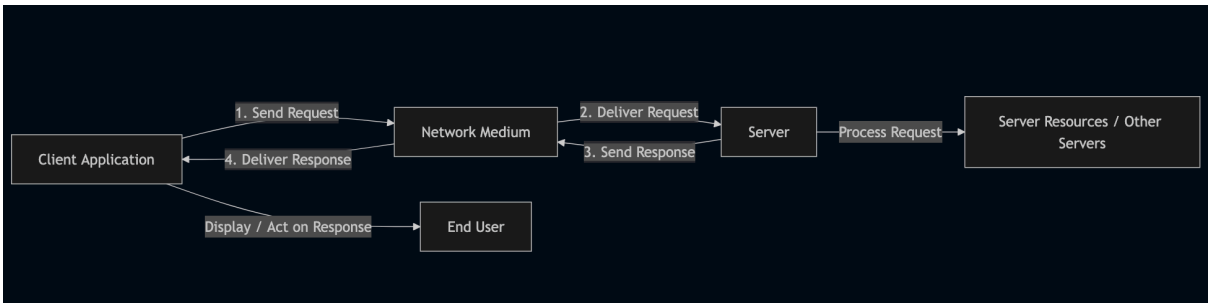
By centralizing decision-making and processing, the application logic ensures consistent behavior, enforces security and compliance rules, and prepares data in a format that clients can directly consume, enabling reliable and scalable server-side operations.



How Does It Work?

In four basic steps, the client/server architecture works like so:

1. The client sends a request to the server using the network medium. The request can be a query, command or message.
2. The server receives the request and processes it according to it’s data and logic. The server may access it’s own resources or other servers to fulfil the request.
3. The server then sends a response back to the client using the network medium. The response can be an acknowledgement, data or an error message such as a 500 HTTP error.
4. The client receives the response and displays it to the user or performs further actions based on it.



References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.

- Covers middleware patterns and design responsibilities, including authentication and validation logic.

Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-Oriented Software Architecture: A System of Patterns.* Wiley.

- Discusses layered architectures and the role of middleware in handling cross-cutting concerns like security and message handling.

Hohpe, G., & Woolf, B. (2004). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions.* Addison-Wesley.

- Provides detailed coverage of message queuing, data translation, and how middleware can enforce validation and security policies.

Fowler, M. (2003). *Patterns of Enterprise Application Architecture.* Addison-Wesley.

- Includes descriptions of how authorization and input validation are typically handled in the service/middleware layers of enterprise applications.

MongoDB Documentation – *Schema Design and Validation*: https://www.mongodb.com/docs/manual/core/schema-validation/

- Explains validation rules at the database level, which tie into server-side request validation.